TECHNICAL REPORT

Secure Smart Hotel IoT Security System Simulation:

Implementing Defence in Depth Through Device-Level

JWT Authentication and Real-Time Intrusion Detection

TABLE OF CONTENTS

1. INTRODUCTION

The rapid proliferation of Internet of Things (IoT) devices across commercial environments has fundamentally transformed how physical systems are managed and monitored. Smart buildings, hotels, and retail facilities increasingly rely on interconnected access control units, surveillance cameras, and HVAC systems to automate operations and improve efficiency. However, this connectivity introduces significant and largely underestimated security risks. The majority of commercially deployed IoT systems operate without dedicated security monitoring, leaving critical infrastructure exposed to adversaries who can compromise devices, manipulate physical systems, and cause significant harm without ever being detected.

This report presents the design, implementation, and security analysis of a Secure Smart Hotel IoT Security System. The system demonstrates that securing the MQTT broker alone is insufficient for a production IoT environment and argues that device-level authentication combined with real-time intrusion detection represents the minimum viable security standard. The project simulates a realistic hotel environment comprising door access control, a reception camera command channel, and an HVAC system, all communicating over a secured MQTT broker with TLS encryption. A lightweight IoT Intrusion Detection System (IoT IDS) monitors all device channels simultaneously, ensuring attacks are not merely blocked but detected, alerted, and visible in real time.

2. IOT SECURITY LANDSCAPE

The IoT security threat landscape is characterised by a persistent gap between the pace of device deployment and the maturity of security practices applied to those deployments. Palo Alto Networks (2024) identify limited visibility and monitoring as one of the most critical IoT security risks, noting that most IoT devices are deployed with little to no built-in monitoring, often failing to generate logs or support telemetry that gives operators meaningful insight into their security status. This creates a condition where a compromised device may remain undetected indefinitely, delaying incident response and enabling persistent adversarial access.

The MQTT protocol, which underpins the majority of IoT messaging systems, is particularly exposed to exploitation. Javed et al. (2022) demonstrate that existing intrusion detection systems have historically lacked support for IoT communication protocols such as MQTT, leaving devices exposed to crafted and malformed packet attacks, authentication bypass exploits, and unauthorised topic publication. Authentication bypass vulnerabilities, in which unencrypted broker communications allow credential theft and broker-level exploitation, are extensively documented in the academic literature. Shingala (2019) further identifies that JWT tokens transmitted without TLS encryption are vulnerable to interception, establishing that the combination of TLS at the transport layer and JWT at the device authentication layer constitutes the minimum credible security baseline for any MQTT-based IoT deployment.

3. THE MONITORING GAP IN COMMERCIAL IOT DEPLOYMENTS

The absence of security monitoring in commercial IoT deployments is not a theoretical concern; it has produced catastrophic real-world consequences. The 2013 Target data breach remains one of the most extensively studied cybersecurity incidents in history. The US Senate Commerce Committee (2014) concluded that the attackers gained their initial foothold through a third-party HVAC vendor, Fazio Mechanical Services, whose weak security practices allowed the installation of credential-stealing malware. This malware provided the attackers with network access credentials that were subsequently used to pivot into Target's internal payment systems, ultimately compromising approximately 40 million payment card records. Critically, the Senate report notes that Target's own automated intrusion detection systems raised multiple alerts during the active breach phase, alerts that went unacknowledged and unacted upon by the security team. This incident demonstrates that the absence of adequate security monitoring transforms a blockable attack into an uncontrolled breach.

The scale of the monitoring gap in contemporary IoT deployments remains significant. i-scoop.eu (2021), citing Gartner Research Director Saniye Burcu Alaybeyi, reports that the vast majority of IoT devices, whether connected via wired switches, Bluetooth, or Wi-Fi, are unmonitored and resource-constrained, lacking the capacity to run traditional security agents or be scanned remotely. SentinelOne (2024) corroborate this position, stating that most IoT deployments function without proper security monitoring solutions, producing high volumes of operational data while failing to log security events, leaving security teams unable to identify active attacks before they reach a critical status.

This monitoring gap constitutes the central security problem this project is designed to address. A system that blocks attacks silently, generating no log, no alert, and no opportunity for investigation, provides fundamentally incomplete security. An adversary who is blocked but undetected retains the ability to probe the system indefinitely, adjusting their approach with each failed attempt until a successful compromise is achieved.

4. SYSTEM ARCHITECTURE AND OPERATIONAL FLOW

The system implements a three-layer defence in depth architecture. Sharma et al. (2023) describe defence in depth as the creation of multiple independent security hurdles that an attacker must overcome sequentially, ensuring that the failure of any single layer does not result in total system compromise. The three layers are as follows.

Layer 1: TLS Encryption on the MQTT Broker

All device communications are encrypted using Transport Layer Security (TLS) on port 8883. The Mosquitto broker is configured with a self-signed Certificate Authority, a server certificate, and a password-authenticated user account for each device. This layer ensures that traffic between devices and the broker cannot be intercepted, read, or tampered with in transit, directly mitigating man-in-the-middle attacks and credential eavesdropping.

Layer 2: JWT Verification at Device Controller Level

Each device controller independently verifies JWT tokens on every incoming command before executing any action. Shingala (2019) identifies this distributed authentication model as consistent with the approach mandated by Google Cloud IoT for MQTT clients, where JWT verification at the device level ensures that valid broker credentials alone are insufficient to achieve device control. The controller makes the access decision autonomously. The JWT provides the cryptographic evidence; the controller acts on that evidence independently, without relying on any central authority. This is the critical architectural distinction that separates this system from deployments that rely solely on broker-level security.

Layer 3: Lightweight IoT IDS

The security_monitor(IDS).py component subscribes to MQTT topics across all three device channels simultaneously, monitoring for denial events, forged token attempts, and dangerous HVAC setpoint injections. This layer is discussed in detail in Section 7.

Operational Flow

The complete flow of both legitimate and adversarial activity through the system is illustrated in Figure 1 below.

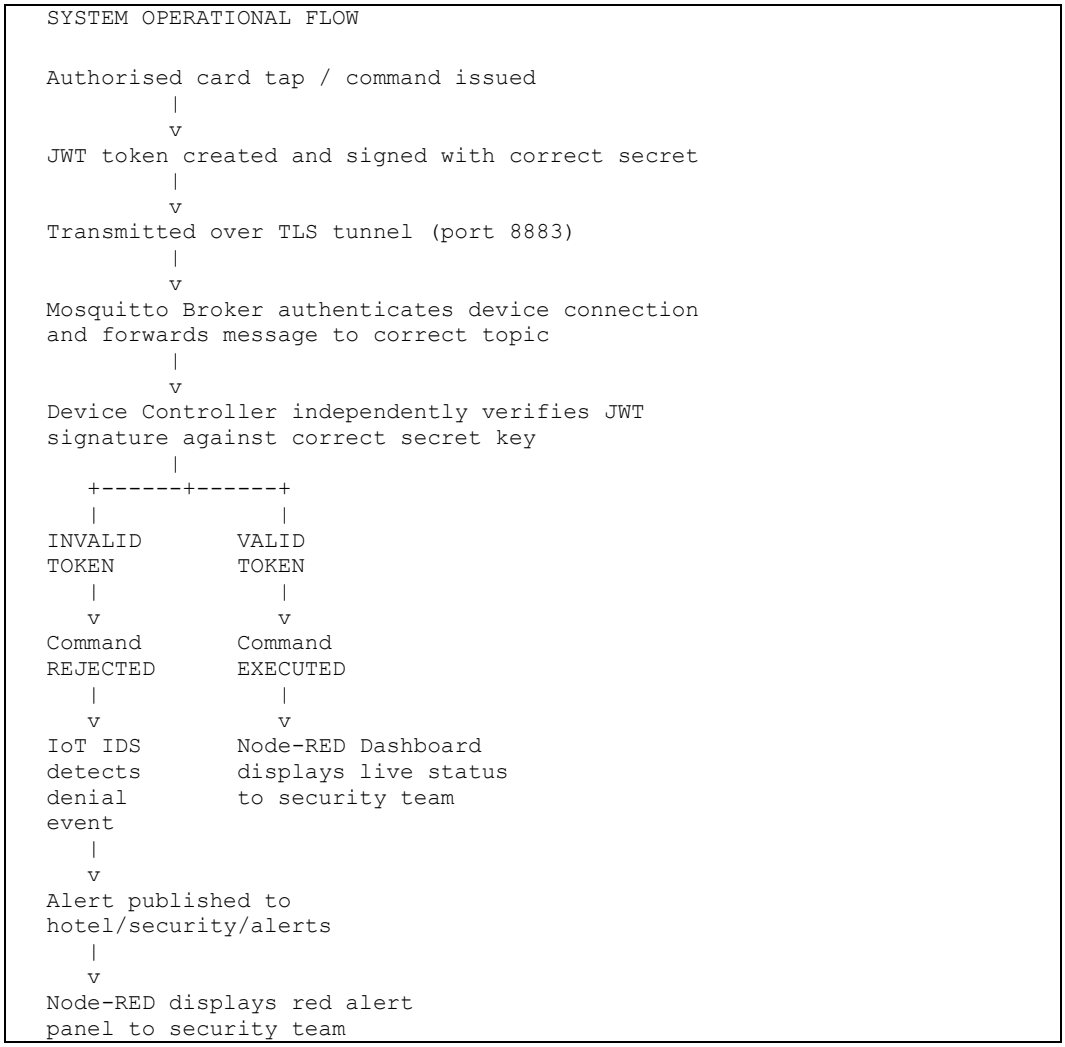

Figure 1: System operational flow showing both authorised and adversarial command paths.

The MQTT broker intentionally forwards all well-formed messages regardless of payload content, consistent with its role as a stateless transport layer. Payload authenticity is fully delegated to each device controller, implementing a Zero Trust model in which every device independently verifies every command. Sharma et al. (2023) confirm that Zero Trust principles, which advocate for the elimination of implicit trust and continuous per-request verification, are particularly effective in IoT environments where traditional perimeter-based security models have consistently proven inadequate.

5. COMPARISON TO REAL-WORLD DEPLOYMENTS

The system components correspond directly to their real-world production equivalents. The door_controller(ACU).py script simulates an Access Control Unit (ACU), the embedded controller inside a physical card reader or electronic lock that makes grant or deny decisions. In production, this logic executes on hardened embedded firmware rather than a Python script, but the security model is identical: JWT verification at device level, autonomous decision, with no dependence on a central authority.

The reception_camera1.py script simulates the onboard command-processing firmware of an IP surveillance camera, which in a production environment would accept or reject pan, tilt, and reboot commands based on verified credentials. The hvac.py script simulates a Building Management System (BMS) HVAC controller, the embedded unit responsible for temperature regulation, which in production would enforce setpoint safety limits regardless of the instruction source.

The security_monitor(IDS).py component represents the most significant architectural contribution of this project in relation to real-world practice. In a production smart building, this function would be fulfilled by a dedicated IoT security platform such as Claroty or Armis, enterprise-grade systems that passively monitor all device communication channels, detecting anomalous behaviour and publishing alerts to a centralised Security Operations Centre (SOC). Claroty (2025) describes its core function as providing analysts with real-time threat detection and the ability to assess risk posture across all connected devices simultaneously. The security_monitor(IDS).py implements this same function at academic scale, subscribing to MQTT topics, applying rule-based detection logic, and publishing alerts, demonstrating the principle that underpins every enterprise IoT IDS.

In production, the Python scripts would be replaced by compiled firmware and dedicated hardware appliances. The Mosquitto broker would be hosted on a hardened server or a managed cloud IoT platform such as AWS IoT Core or HiveMQ. The IoT IDS would feed alerts into a full SIEM platform such as Splunk or IBM QRadar for log correlation, storage, and incident response workflow management. The Node-RED dashboard would be replaced by a full SOC monitoring console. The core security architecture, TLS at transport layer, JWT at device layer, and IoT IDS at observation layer, remains architecturally identical across both academic and production implementations.

6. ATTACK SIMULATION: JWT INJECTION ATTACK SIMULATOR

The hacker.py script is a JWT Injection Attack Simulator replicating the final injection phase of a documented MQTT-based attack chain. The adversary is assumed to have already obtained valid broker credentials, consistent with documented real-world techniques including firmware credential extraction and man-in-the-middle interception (Javed et al., 2022). The script generates a JWT signed with an incorrect secret key and publishes it to all three device command topics without delays, print statements, or academic commentary, replicating the concise, intent-driven behaviour of real adversarial tooling.

This attack is realistic because the MQTT broker, acting as a transport layer, forwards the message to the device controller without inspecting its payload. The controller then performs JWT verification, discovers the invalid signature, and rejects the command, demonstrating that device-level authentication blocks the attack even when broker access has been fully compromised.

It is important to acknowledge that IoT attacks exist in many forms beyond this scenario. Real-world attack vectors include brute-force credential attacks, replay attacks, man-in-the-middle interception, denial of service flooding, and firmware extraction (Javed et al., 2022). This simulation deliberately focuses on forged JWT token injection to demonstrate device-level JWT verification as an independent second security layer, the specific and original security contribution of this project. The simulation begins at the post-reconnaissance phase not as a simplification but as a deliberate choice to isolate and prove the value of the architectural security layer being demonstrated.

7. THE IOT IDS: CLOSING THE VISIBILITY GAP

The security_monitor(IDS).py component functions as a lightweight IoT IDS agent implementing rule-based detection logic. Gyamfi and Jurcut (2022) present a comprehensive review of network intrusion detection systems for IoT, confirming that detection components operating at the protocol communication layer, monitoring published MQTT messages for anomalous patterns, represent a well-established and effective approach to IoT threat detection. Javed et al. (2022) further establish that an efficient intrusion detection system for MQTT-based applications was, at the time of their writing, still a missing piece in the IoT security landscape, a gap that the IoT IDS in this project directly addresses at the application layer.

The IoT IDS subscribes to MQTT topics across all three device channels. When the door controller rejects a forged JWT, the IDS detects the denial event and immediately publishes an alert. When a camera command arrives with invalid credentials, the IDS raises an alert concurrently with the controller rejection. When an HVAC setpoint exceeds the safe threshold, the IDS detects and alerts simultaneously. All alerts are forwarded to the Node-RED dashboard, where the security team receives immediate, actionable visibility of every attack attempt.

The architectural distinction between controller and IoT IDS is fundamental: the controller stops the attack and the IoT IDS makes sure someone knows it happened. Without this monitoring layer, every attack is blocked silently, no log is created, no alert is raised, and no investigation is possible. The attacker can probe the system indefinitely without anyone knowing. This is precisely the condition that allowed the Target breach to escalate from a network intrusion to a catastrophic data compromise (US Senate Commerce Committee, 2014). This project ensures that condition cannot arise.

8. ALIGNMENT WITH INDUSTRY STANDARDS

The architecture implemented in this project aligns with two major international frameworks governing IoT and information system security.

NIST SP 800-53 Revision 5

NIST Special Publication 800-53 Revision 5 (NIST, 2020), published by the National Institute of Standards and Technology, provides a comprehensive catalogue of security and privacy controls covering all computing platforms including IoT devices. The continuous monitoring control family (CA-7) within this framework mandates that organisations implement ongoing monitoring of security controls to ensure their continued effectiveness, and that security-relevant events are detected, reported, and responded to in a timely manner. This directly corresponds to the function of the IoT IDS in this project, which monitors all device channels continuously and generates real-time alerts for every detected attack event.

IEC 62443

The IEC 62443 series of standards, which IEC PAS 62443-1-6 (IEC, 2025) now explicitly extends to Industrial IoT environments including smart building systems, establishes cybersecurity requirements for industrial automation and control systems. The standard's defence-in-depth requirements mandate multiple independent security layers, precisely the architecture implemented here. Its zone-and-conduit model, in which device communications are monitored and controlled across defined security boundaries, mirrors the layered approach of this project, where each controller enforces its own independent security boundary and the IoT IDS monitors all channels simultaneously.

Together, these frameworks confirm that the monitoring layer implemented in this project is not an academic exercise but a compliance requirement for any production IoT deployment operating within a regulated environment, one that most commercial smart building deployments currently fail to meet.

9. CONCLUSION

This project demonstrates that broker-level security alone is fundamentally insufficient for production IoT deployments. Even when an adversary obtains valid broker credentials, device-level JWT verification provides a critical second security layer that blocks unauthorised commands autonomously at each controller, without dependence on any central authority. The lightweight IoT IDS then closes the visibility gap that would otherwise leave every blocked attack silently undetected, with no log, no alert, and no investigation possible. Together, TLS encryption at the transport layer, JWT verification at the device controller layer, and real-time intrusion detection at the observation layer implement a defence in depth architecture that addresses the monitoring gap identified by Gartner, mandated by NIST SP 800-53 and IEC 62443, and demonstrated as critically necessary by the 2013 Target breach. Most commercial IoT systems leave the monitoring layer entirely unguarded. This project demonstrates what best practice looks like when it is not.

REFERENCES

Claroty (2025) Claroty Named a Leader in IoT Security by Leading Independent Research Firm [online]. Available at: https://claroty.com/press-releases/claroty-named-a-leader-in-iot-security-by-leading-independent-research-firm [Accessed: 6 April 2026].

Gyamfi, E. and Jurcut, A. (2022) 'Intrusion Detection in Internet of Things Systems: A Review on Design Approaches Leveraging Multi-Access Edge Computing, Machine Learning, and Datasets', Sensors, 22(10), p.3744 [online]. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC9143513/ [Accessed: 6 April 2026].

i-scoop.eu (2021) IoT security: the majority of IoT devices is not monitored in real time [online]. Available at: https://www.i-scoop.eu/iot-security-majority-iot-devices-not-monitored-real-time/ [Accessed: 6 April 2026].

IEC (2025) IEC PAS 62443-1-6:2025 Security for industrial automation and control systems: Application of the 62443 series to the Industrial Internet of Things (IIoT) [online]. Available at: https://webstore.iec.ch/en/publication/102885 [Accessed: 6 April 2026].

Husnain, M., Hayat, K., Cambiaso, E., Fayyaz, U.U., Mongelli, M., Akram, H., Abbas, S.G. and Shah, G.A. (2022) 'Preventing MQTT Vulnerabilities Using IoT-Enabled Intrusion Detection System', Sensors, 22(2), p.567 [online]. Available at: https://pmc.ncbi.nlm.nih.gov/articles/PMC8779830/ [Accessed: 6 April 2026].

National Institute of Standards and Technology (NIST) (2020) Security and Privacy Controls for Information Systems and Organizations: Special Publication 800-53 Revision 5 [online]. Available at: https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final [Accessed: 6 April 2026].

Palo Alto Networks (2024) Top 10 IoT Security Issues: Challenges and Solutions [online]. Available at: https://www.paloaltonetworks.ca/cyberpedia/iot-security-issues [Accessed: 6 April 2026].

Gajbhiye, B., Jain, S. and Goel, O. (2023) 'Defense in Depth Strategies for Zero Trust Security Models', Darpan International Research Analysis, 11(1), pp.27–39 [online]. Available at: https://dira.shodhsagar.com/index.php/j/article/view/70 [Accessed: 6 April 2026].

Shingala, K. (2019) 'JSON Web Token (JWT) based client authentication in Message Queuing Telemetry Transport (MQTT)', arXiv [online]. Available at: https://arxiv.org/abs/1903.02895 [Accessed: 6 April 2026].

SentinelOne (2024) Top 10 IoT Security Risks and How to Mitigate Them [online]. Available at: https://www.sentinelone.com/cybersecurity-101/data-and-ai/iot-security-risks/ [Accessed: 6 April 2026].

US Senate Commerce Committee (2014) A Kill Chain Analysis of the 2013 Target Data Breach [online]. Available at: https://www.commerce.senate.gov/wp-content/uploads/media/doc/2014%200325%20Target%20Kill%20Chain%20Analysis.pdf [Accessed: 6 April 2026].